Del inventario al despliegue: modelos de aprendizaje automático para priorizar el mantenimiento de una red GPON

Juan Ignacio Forzani**; Alejo Racca**; Agustín Cesano**

UNRaf, Universidad Nacional de Rafaela, Santa Fe, Argentina.
Aprendizaje Automático y Grandes Datos (IC)

(**) Autores Corresp.: forzanijuan@gmail.com; alejoracca07@gmail.com; agustincesano09@gmail.com

Resumen

Las empresas de fibra óptica registran el inventario de los equipos de sus clientes (ONT), pero rara vez lo analizan para planificar el mantenimiento. Este trabajo integra las cuatro etapas de un proyecto de aprendizaje automático sobre el inventario de 5.114 ONT de una red GPON de Santa Fe y el listado de su central (OLT). El propósito fue apoyar dos decisiones: qué puertos actualizar primero y a qué puerto conectar a un cliente nuevo. Tras el análisis exploratorio, se compararon cuatro algoritmos de regresión para estimar el porcentaje de equipos con *firmware* fuera de estándar en los 301 puertos PON, y cuatro clasificadores, dos ensambles y *K-Means* para anticipar si un cliente queda con la señal en riesgo, con validación agrupada por puerto. La regresión *Ridge* logró un R^2 de 0,59 en prueba y la regresión logística, un ROC-AUC de 0,70 y un PR-AUC de 0,39, igual que una regla basada en la señal de los vecinos del puerto: el riesgo depende de la fibra compartida, no del equipo. Ambos modelos se serializaron y se desplegaron en una aplicación web. Cumplen sus metas, pero sirven para ordenar prioridades, no para decidir caso por caso.

Palabras claves: Redes GPON, Aprendizaje Automático, Regresión *Ridge*, Regresión Logística, Validación Agrupada, Despliegue de Modelos.

1. Introducción

Las redes ópticas pasivas con capacidad de *gigabit* (GPON) llevan la fibra hasta el hogar desde una terminal de línea óptica (OLT), ubicada en la central y dividida en tarjetas y puertos PON. Cada puerto atiende, por un mismo tendido, a decenas de terminales de red óptica (ONT): el módem que el cliente tiene en su casa. Para un proveedor regional, dos tareas consumen buena parte del mantenimiento: actualizar el *firmware* de los equipos, que se hace de a un puerto entero dentro de una ventana de mantenimiento, y conectar a cada cliente nuevo en un puerto donde la señal le llegue con margen. La norma G.984.2 fija para los equipos de clase B+ una sensibilidad mínima de −27 dBm [1]: un cliente que recibe menos de −25 dBm queda a solo 2 dB de perder el servicio.

El proyecto usó datos reales de una empresa de fibra óptica de la región. Mantener el *firmware* al día corrige fallas de seguridad y de compatibilidad, pero cada actualización deja sin servicio a un puerto entero durante la ventana; y con poco margen de señal, un empalme degradado alcanza para cortar el servicio. Hoy ambas decisiones se toman con planillas y con la experiencia del personal técnico.

El aprendizaje automático se aplica a las redes ópticas desde hace más de una década: Zibar *et al.* [2] y Khan *et al.* [3] muestran cómo pasó del procesamiento de la señal en el receptor a la gestión de la red. Las revisiones de Musumeci *et al.* [4] y de Mata *et al.* [5] agrupan sus usos en estimación de la calidad de transmisión, gestión de fallas, predicción de tráfico y monitoreo del desempeño, y Rafique y Velasco [6] proponen arquitecturas para automatizar la red con esos modelos. En casi todos los casos se entrena con series de monitoreo (potencia óptica, tasa de errores, alarmas) de redes troncales o metropolitanas y con grandes volúmenes de datos. Con tablas pequeñas conviene comparar modelos lineales regularizados como *Ridge* [7], [8] con ensambles de árboles como *Random Forest* [9] o *Gradient Boosting* [10], más flexibles pero propensos al sobreajuste. Con clases desbalanceadas conviene ajustar los pesos de clase o el umbral [11], y la curva precisión-*recall* es más informativa que la ROC [12]. Dos errores frecuentes invalidan los resultados: la fuga de información, cuando una entrada contiene la respuesta [13], y la validación aleatoria sobre datos agrupados, que reparte entre entrenamiento y prueba observaciones casi idénticas [14]. Por último, llevar un modelo a producción agrega costos que no se ven en un *notebook*: dependencias, datos que cambian y monitoreo [15], [16].

Esos trabajos suponen datos de monitoreo continuos. Un proveedor regional, en cambio, suele tener solo el inventario de equipos (una foto estática, sin fechas ni fallas) y, a lo sumo, un listado puntual de la OLT. No encontramos trabajos que evalúen qué se puede aprender honestamente con esa información ni

que lleven los modelos a una herramienta que el personal técnico pueda usar sin programar.

El objetivo general fue convertir el inventario de una red GPON real en herramientas de decisión para el mantenimiento, en cuatro etapas: (i) limpiar y explorar el inventario (TP1); (ii) estimar el porcentaje de equipos con *firmware* fuera de estándar de cada puerto PON, con la meta de alcanzar en prueba un R^2 de al menos 0,45, un R^2 ajustado de al menos 0,30 y un error absoluto medio menor que el de predecir el promedio (TP2); (iii) anticipar si un cliente queda con la señal en riesgo antes de medirla, con un ROC-AUC de al menos 0,68 y un PR-AUC de al menos 0,22, el doble de la tasa base (TP3), y (iv) desplegar los modelos elegidos en una aplicación web (TP4).

La sección 2 describe los datos y los métodos; la 3, los resultados de los modelos y de la aplicación; la 4 los discute junto con sus limitaciones, y la 5 resume las conclusiones y el trabajo futuro.

2. Metodología

2.1 Conjunto de datos

Se trabajó con dos archivos entregados por una empresa de telecomunicaciones de la región. El inventario tiene 5.114 filas, una por ONT, y 13 columnas: ubicación (localidad, tarjeta, puerto e identificador), marca, modelo, versión de *firmware* y cinco atributos de *hardware* (puertos LAN, tecnología Wi-Fi, módulo óptico, puertos telefónicos y salida de TV). Cubre 9 localidades, 23 tarjetas y 301 puertos PON, y es una foto estática: no trae fechas, estados, potencias ni fallas. El segundo archivo es el listado de la OLT del 29/10/2026, con 5.804 equipos, su estado, las potencias transmitida y recibida y la distancia de fibra hasta la central; 5.052 de ellos cruzan con el inventario por localidad, *slot*, PON e identificador. La Fig. 1 resume el flujo del proyecto y la Tabla 1, cómo se usó cada fuente en los dos problemas supervisados.

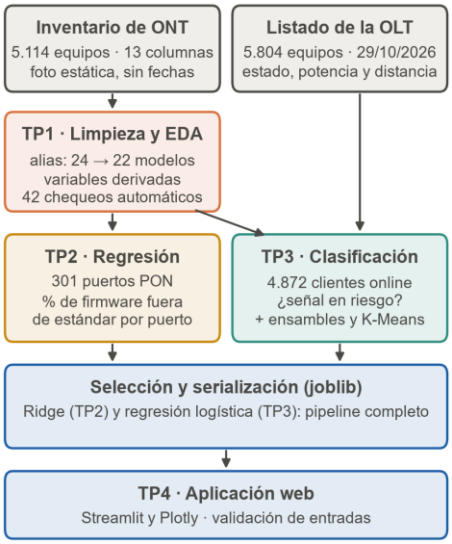


Fig. 1: Flujo del proyecto: fuentes de datos, etapas (TP1 a TP4) y modelos que llegan a la aplicación.

Tabla 1. Problemas supervisados del proyecto.

	TP2 (regresión)	**TP3 (clasificación)**
Unidad	Puerto PON	Cliente online
Filas	301	4.872
Objetivo	% de equipos con *firmware* fuera de estándar	Señal < −25 dBm (11,2 %)
Entradas	Ubicación, tamaño del puerto, % con TV, % modernos y mezcla de modelos	Ubicación, equipo, *firmware*, distancia y señal de los vecinos
Excluidas	Todo lo derivado de la versión de *firmware*	Potencia, estado y nivel de señal del propio equipo
Partición	Aleatoria 80/20 y *KFold* de 5	Agrupada por PON (5 particiones)

2.2 Preprocesamiento y análisis exploratorio (TP1)

En el TP1 se normalizaron los nombres de las localidades, se consolidaron los alias de modelo (por ejemplo, «546M» y «HG8546M»), lo que redujo los 24 nombres a 22 equipos físicos, y se armaron claves únicas de tarjeta y de puerto. Se derivaron variables por equipo: cantidad de equipos del puerto, posición relativa del identificador dentro del puerto, generación del *firmware* (el número que sigue a la «V» de la versión), equipo moderno (cuatro puertos *gigabit* o Wi-Fi 5/6) y *firmware* fuera de estándar, definido como una versión distinta de la más frecuente (la moda) de su modelo, una definición de negocio que la empresa puede reemplazar. Luego se agregaron por puerto (proporción de equipos modernos, con TV y de cada modelo, y cantidad de modelos y de versiones). 42 chequeos automáticos (cantidades, duplicados y consistencia con la tabla de equivalencias de *hardware*) verifican que el procesamiento reproduce

siempre las mismas cifras.

El análisis exploratorio dejó una advertencia que condicionó todo lo que siguió: los cinco atributos de *hardware* son una función determinística del modelo, de modo que un clasificador que los use para predecir el modelo, o al revés, «acierta» sin aprender nada. Es la fuga de información que describen Kaufman *et al.* [13]. Por eso cada problema supervisado declara qué variables quedan prohibidas como entrada (Tabla 1).

2.3 Regresión: firmware por puerto (TP2)

El objetivo es el porcentaje de equipos con *firmware* fuera de estándar de cada puerto PON, que indica cuántos equipos se actualizarían si se interviene ese puerto. Se excluyeron como entradas la proporción de equipos con *firmware* de generación 3 y la cantidad de versiones, porque se calculan a partir de la misma versión que define la respuesta. De los 22 modelos solo entran las proporciones de los cinco más usados: con todos, el R^2 ajustado de prueba cae a −0,04 porque quedan 38 predictores. Los puertos se dividieron en entrenamiento (240) y prueba (61) con semilla 42, y los hiperparámetros se eligieron con *GridSearchCV* y *KFold* de cinco particiones sobre el entrenamiento (Tabla 2). La codificación *one-hot* de localidad y *slot* y el escalado van dentro de un *pipeline* de scikit-learn [17], de modo que nada se ajusta con datos de prueba.

Cada entrada tiene un motivo. La localidad y la tarjeta capturan diferencias de gestión entre zonas; la cantidad de equipos indica el tamaño del puerto; los porcentajes de equipos con salida de TV y de equipos modernos describen su mezcla tecnológica, y la cantidad de modelos distintos mide cuán heterogéneo es. Las proporciones de los cinco modelos más usados (EG8141A5, EG8143A5, CN1001, CN2002, RDC143) aportan la composición sin multiplicar los predictores. Como el estándar se define por modelo, la mezcla de modelos es la vía legítima por la que el modelo puede aprender sin mirar la versión de cada equipo.

Se compararon la regresión lineal como línea base, *Ridge* [7], *Random Forest* [9] y *Gradient Boosting* [10] contra un *DummyRegressor* que predice el promedio. Las métricas fueron MSE, RMSE, MAE, R^2 y R^2 ajustado, en entrenamiento, validación cruzada y prueba. El R^2 ajustado penaliza la cantidad de predictores p para n observaciones:

$$R^2_{aj} = 1 - (1 - R^2)(n - 1)/(n - p - 1) \quad (1)$$

Con $p = 21$ predictores efectivos y $n = 61$ puertos de prueba, la penalización es fuerte. No se usó el error porcentual (MAPE) porque 26 puertos tienen 0 % de equipos fuera de estándar y el cociente se indefine.

Tabla 2. Hiperparámetros explorados con *GridSearchCV*; en negrita, el valor elegido (∞: sin límite de profundidad).

Modelo	Espacio explorado
Ridge (TP2)	α ∈ {0,1; **1**; 10}
Random Forest (TP2)	prof. ∈ {∞; **8**}, mín. hoja ∈ {1; **3**; 5}
Gradient Boosting (TP2)	tasa ∈ {**0,05**; 0,1}, prof. ∈ {**2**; 3}
Reg. logística (TP3)	C ∈ {**0,01**; 0,1; 1; 10}
Árbol (TP3)	prof. ∈ {3; 5; **8**; ∞}, mín. hoja ∈ {10; **50**}
KNN (TP3)	vecinos ∈ {15; 31; 51; **101**}, peso ∈ {unif.; **dist.**}
Naive Bayes (TP3)	suavizado ∈ {10^{-9}; 10^{-6}; 10^{-3}; **0,1**}
Random Forest (TP3)	prof. ∈ {∞; **10**}, mín. hoja ∈ {1; **5**; 20}
Gradient Boosting (TP3)	tasa ∈ {**0,05**; 0,1}, prof. ∈ {**2**; 3}

2.4 Clasificación y agrupamiento: señal en riesgo (TP3)

El TP3 se planteó para evitar la fuga del TP1: ¿se puede anticipar que un cliente queda con la señal al límite antes de medirla? Se usaron los 4.872 equipos online que cruzan con el inventario y tienen lectura válida; la clase positiva es una potencia recibida menor que −25 dBm (548 casos, 11,2 %), 2 dB por encima de la sensibilidad de la clase B+ [1]. La potencia, el estado y el nivel de señal del propio equipo quedaron prohibidos como entradas. Sí entra la señal media de los vecinos del mismo puerto sin contar al equipo (*vec_rxp*), porque ya está medida cuando se instala un cliente nuevo. Para el cliente i de un puerto P con n_P equipos online:

$$vec_rxp_i = (\Sigma_{j \in P}\, RXP_j - RXP_i) / (n_P - 1) \quad (2)$$

Los 8 clientes que son los únicos online de su puerto no tienen vecinos: su valor se imputa con la mediana dentro del *pipeline*, que además codifica las variables categóricas con *one-hot* y escala las numéricas.

Como los equipos de un puerto comparten la fibra, la partición se hizo con *StratifiedGroupKFold* de cinco particiones agrupando por puerto [14]: una de ellas (973 clientes de 63 puertos) quedó como prueba y la validación cruzada se hizo con la misma estrategia sobre el resto. Se compararon la regresión logística (línea base), un árbol de decisión, KNN y *Naive Bayes* gaussiano contra un *DummyClassifier* que respeta la proporción de clases y contra una regla sin aprendizaje que ordena a los clientes por la señal de sus vecinos.

Para el desbalance se usaron pesos de clase balanceados donde existen y, en KNN y *Naive Bayes*, un umbral elegido en validación cruzada. Como extensión se entrenaron *Random Forest* y *Gradient Boosting*, y *K-Means* con *k* de 2 a 8, elegido por silueta [18] y Davies-Bouldin y comparado con la clase real mediante el índice de Rand ajustado (ARI). Se calcularon accuracy, precisión, *recall*, F1, Kappa, ROC-AUC y PR-AUC; la búsqueda de hiperparámetros maximizó el PR-AUC.

2.5 Estrategia de despliegue (TP4)

Se desplegó un modelo por problema, elegido con dos criterios fijados antes de mirar la prueba: el mejor desempeño en validación cruzada y una brecha chica entre entrenamiento y validación, porque un modelo que memoriza es frágil frente a datos nuevos. En el TP3 la elección se hizo entre los cuatro clasificadores individuales que pide la consigna; los ensambles quedaron como control. Cada modelo se reentrenó con el mismo conjunto de entrenamiento del informe y se serializó con *joblib* junto con su preprocesamiento: el archivo guarda el *pipeline* completo, las columnas de entrada en orden y la versión de scikit-learn. Un control automático verifica que el modelo guardado reproduce exactamente las métricas de prueba antes de publicarlo.

La aplicación se construyó con Streamlit [19] y Plotly. Un menú lateral agrupa un resumen del TP1, la metodología y un apartado por modelo, organizado en pestañas que responden una pregunta cada una (Tabla 3). El margen de error que acompaña cada estimación del TP2 es el MAE de prueba (8,8 puntos), y el riesgo del TP3 se expresa de 0 a 100 como el percentil del puntaje del modelo entre los clientes actuales, con un semáforo: menos de 50, conectar; de 50 a 80, revisar la fibra antes; 80 o más, elegir otro puerto. La carpeta de la aplicación es autocontenida: incluye el código, los dos modelos, los datos procesados que muestra (sin números de serie ni direcciones IP) y un archivo requirements.txt con versiones fijas, de modo que corre en cualquier computadora con Python sin acceder a los datos originales.

Tabla 3. Elementos que pide la consigna para la aplicación y cómo se cumplen.

Requisito	Implementación
Título y descripción	Encabezado de cada apartado con el problema y el modelo usado
Entrada de datos	Selectores y deslizadores que parten de los valores de un puerto o cliente real
Salida	% estimado con su margen de error (TP2); riesgo de 0 a 100 y un veredicto en palabras (TP3)
Validación	Rangos acotados y avisos cuando un valor o una combinación queda fuera de lo que vio el modelo
Contexto	Posición del caso frente a los puertos o clientes reales, importancia de variables y desempeño
Extensiones	Dos modelos, gráficos interactivos, explicabilidad y descarga de resultados en CSV

2.6 Reproducibilidad

Todo el procesamiento usa la semilla 42 y corre desde cero con un único comando (python run_all.py), que encadena la preparación de los datos, los dos trabajos, la serialización de los modelos, la carpeta de la aplicación y este artículo; los *notebooks* del TP2 y del TP3 se ejecutan de principio a fin como parte de la verificación. Cada cifra de los informes, de la aplicación y de este artículo se lee de un único archivo de métricas generado por el código: ninguna se copió a mano.

3. Resultados

3.1 Análisis exploratorio

La Fig. 2 resume los hallazgos del TP1 que motivaron los dos problemas supervisados. Toda la red es de una sola marca (Huawei) y tres modelos concentran el 69,8 % de los equipos (Fig. 2a). El 97,2 % de las ONT solo tiene Wi-Fi 4 y el 95,2 % tiene al menos un puerto LAN de 100 Mb/s: el límite del servicio está en el módem del cliente, no en la fibra. El 28,9 % de los equipos tiene el *firmware* fuera de estándar, con diferencias grandes entre localidades: en Larrechea llega al 86,8 % (Fig. 2b). Además, 6 de las 23 tarjetas atienden menos de 120,5 equipos (el 25 % menos ocupado), una señal de capacidad ociosa. Con ese diagnóstico, el TP1 propuso una meta medible: pasar de 2,7 % a 35,9 % de equipos con Wi-Fi 5 o superior antes del 31/12/2027, reemplazando los 1.698 EG8141A5, el modelo más común.

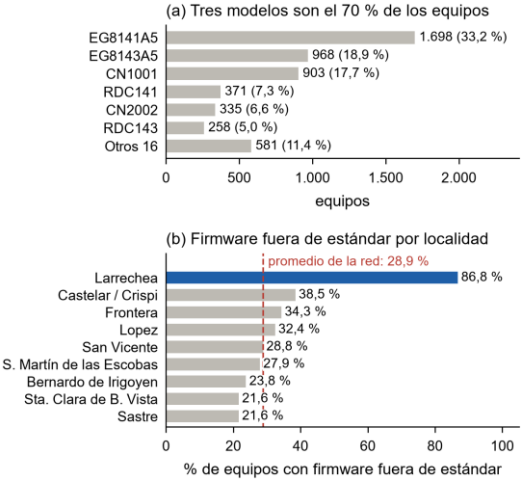


Fig. 2: Análisis exploratorio del inventario (TP1). (a) Equipos por modelo: ninguno de los seis más usados tiene Wi-Fi 5 o 6. (b) Porcentaje de equipos con firmware *fuera de estándar por localidad.*

3.2 Regresión

La Tabla 4 y la Fig. 3 comparan los modelos. Los cuatro superan con amplitud al *Dummy*, cuyo R^2 es nulo. *Random Forest* y *Gradient Boosting* obtienen el R^2 de prueba más alto (0,62 y 0,60), pero su R^2 de entrenamiento (0,76 y 0,73) supera en 0,30 y 0,35 al de validación cruzada: memorizan. La regresión lineal y *Ridge* tienen brechas de 0,09 y 0,08 y el mejor desempeño en validación cruzada; se eligió *Ridge* ($\alpha = 1$), con R^2 0,59, R^2 ajustado 0,38 y MAE 0,088 en prueba, frente a un MAE de 0,128 del *Dummy*. Cumple las tres metas. En términos prácticos, la estimación de cada puerto se equivoca en promedio en 8,8 puntos porcentuales.

Tabla 4. Métricas de regresión (TP2): R^2 en entrenamiento (Ent.), validación cruzada (VC) y prueba; R^2 ajustado y MAE en prueba. En negrita, el modelo elegido.

Modelo	Ent.	VC	Prueba	R^2 aj.	MAE
Dummy	0,00	−0,01	−0,01	−0,01	0,128
Lineal	0,58	0,49	0,59	0,37	0,089
Ridge	**0,57**	**0,49**	**0,59**	**0,38**	**0,088**
Random Forest	0,76	0,46	0,62	0,41	0,085
Gradient Boosting	0,73	0,39	0,60	0,39	0,083

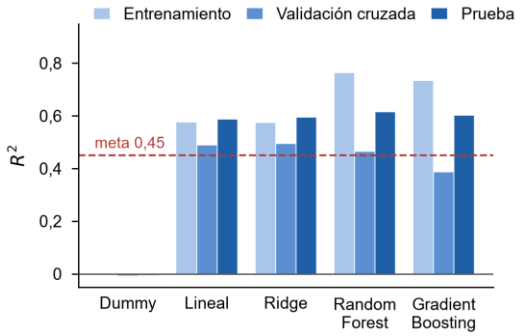


Fig. 3: R^2 de cada modelo de regresión en entrenamiento, validación cruzada y prueba (TP2). La línea roja marca la meta en prueba.

La Fig. 4 muestra el análisis de residuos de *Ridge*. Los puntos siguen la diagonal, pero el modelo sobreestima los puertos con 0 % y subestima los cercanos a 100 %, algo esperable en una variable acotada. Los residuos de prueba no son normales (Shapiro-Wilk, $p = 0{,}008$) ni muestran heterocedasticidad significativa (Breusch-Pagan, $p = 0{,}11$). El error es mayor en los 35 puertos de entrenamiento con cinco equipos o menos (MAE de 15,7 puntos contra 10,1).

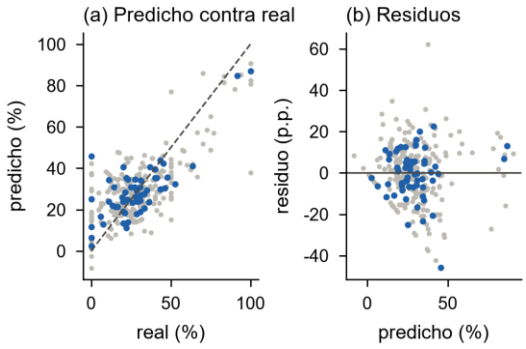


Fig. 4: Residuos de Ridge *(TP2). Gris: entrenamiento fuera de muestra (validación cruzada); azul: prueba. (a) Predicho contra real, con la diagonal de predicción perfecta. (b) Residuos contra predichos.*

La curva de aprendizaje muestra que el error de validación de *Ridge* baja de 16,2 a 14,2 puntos al pasar de 57 a 192 puertos de entrenamiento, con una brecha final de 1,2 puntos frente a 5,0 de *Random Forest*. La curva todavía desciende: con más puertos, por ejemplo de otras redes de la empresa, el modelo mejoraría.

Por permutación (Fig. 5a), las variables más importantes fueron el porcentaje de equipos con TV, la localidad y las proporciones de EG8141A5 y EG8143A5. El porcentaje con TV y la proporción de EG8143A5 tienen una correlación de 0,99 (el EG8143A5 es el modelo con salida de TV), así que su importancia se reparte entre ambas. Sumar variables de la OLT por puerto (porcentaje online, distancia y potencia medianas) no mejoró el modelo: el R^2 de prueba bajó de 0,59 a 0,51. El *firmware* fuera de estándar responde a decisiones administrativas por localidad y por modelo, no al estado físico de la red.

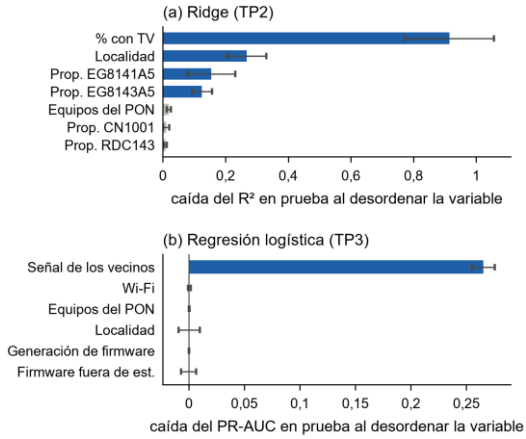


Fig. 5: Importancia por permutación en prueba, con su desvío: (a) caída del R^2 de Ridge *(TP2); (b) caída del PR-AUC de la regresión logística (TP3).*

3.3 Clasificación

La Tabla 5 resume el desempeño en prueba. El *Dummy* tiene una accuracy de 0,889 porque nunca marca a nadie en riesgo: con este desbalance la accuracy engaña, y por eso la selección se basó en ROC-AUC y PR-AUC. La regresión logística (C = 0,01) fue el mejor clasificador individual en validación cruzada (ROC-AUC 0,72 y PR-AUC 0,32) y el de menor brecha con el entrenamiento (0,02). En prueba logró un ROC-AUC de 0,70 y un PR-AUC de 0,39, por encima de las metas. Detectó 60 de los 108 clientes en riesgo (56 % de *recall*) a cambio de 177 falsas alarmas (Fig. 6), con una precisión de 0,25: de cada cuatro clientes marcados, uno está realmente en riesgo.

Tabla 5. Métricas de clasificación (TP3): ROC-AUC en entrenamiento (Ent.) y en prueba (ROC); PR-AUC (PR), *recall* (Rec.) y precisión (Prec.) en prueba (973 clientes, 108 en riesgo). La regla de los vecinos no tiene umbral.

Modelo	Ent.	ROC	PR	Rec.	Prec.
Dummy	0,50	0,50	0,11	0,00	0,00
Regla de vecinos	0,73	0,71	0,40	—	—
Reg. logística	**0,74**	**0,70**	**0,39**	**0,56**	**0,25**
Árbol	0,84	0,71	0,40	0,52	0,23
KNN	1,00	0,66	0,26	0,33	0,34
Naive Bayes	0,73	0,67	0,28	0,36	0,24
Random Forest	0,92	0,68	0,31	0,40	0,35
Gradient Boosting	0,80	0,71	0,43	0,37	0,46

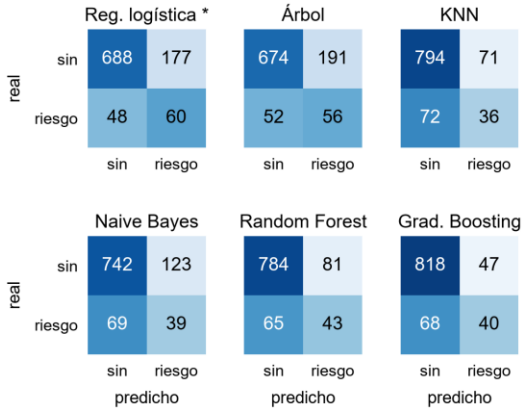


Fig. 6: Matrices de confusión en prueba (TP3). Filas: clase real; columnas: clase predicha. El asterisco marca el modelo elegido.

El hallazgo central es que la regla de los vecinos, sin aprendizaje, obtiene el mismo desempeño (ROC-AUC 0,71 y PR-AUC 0,40; Fig. 7), y la importancia por permutación lo confirma (Fig. 5b): desordenar *vec_rxp* hace caer el PR-AUC en 0,27, mientras que ninguna otra variable lo reduce en más de 0,001. La señal de un cliente depende de la fibra que comparte con su puerto; el modelo del equipo, el *firmware* y la distancia casi no agregan información. El árbol y KNN sobreajustan (brechas de 0,17 y 0,31) y *Naive Bayes* queda por debajo. Los ensambles tampoco mejoran: *Random Forest*, el mejor de todos en validación cruzada, pasa de 0,92 en entrenamiento a 0,675 en prueba, debajo de la meta, y *Gradient Boosting*, aunque alcanza el mayor PR-AUC (0,43), detecta solo 40 de los 108 casos con su umbral.

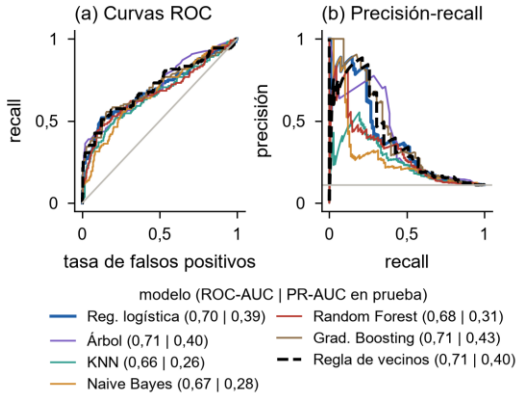


*Fig. 7: Curvas ROC (a) y precisión-*recall *(b) en prueba (TP3). La regla de los vecinos (negro, discontinua) rinde igual que la regresión logística (azul).*

El umbral de decisión define el compromiso entre encontrar clientes en riesgo y no generar falsas alarmas (Fig. 8). Si se prioriza no perder clientes en riesgo, el umbral que da un *recall* del 80 % en validación cruzada (0,36) eleva el *recall* de prueba a 0,74, pero marca a 547 clientes con una precisión de 0,15. La aplicación deja elegir ese compromiso con un deslizador.

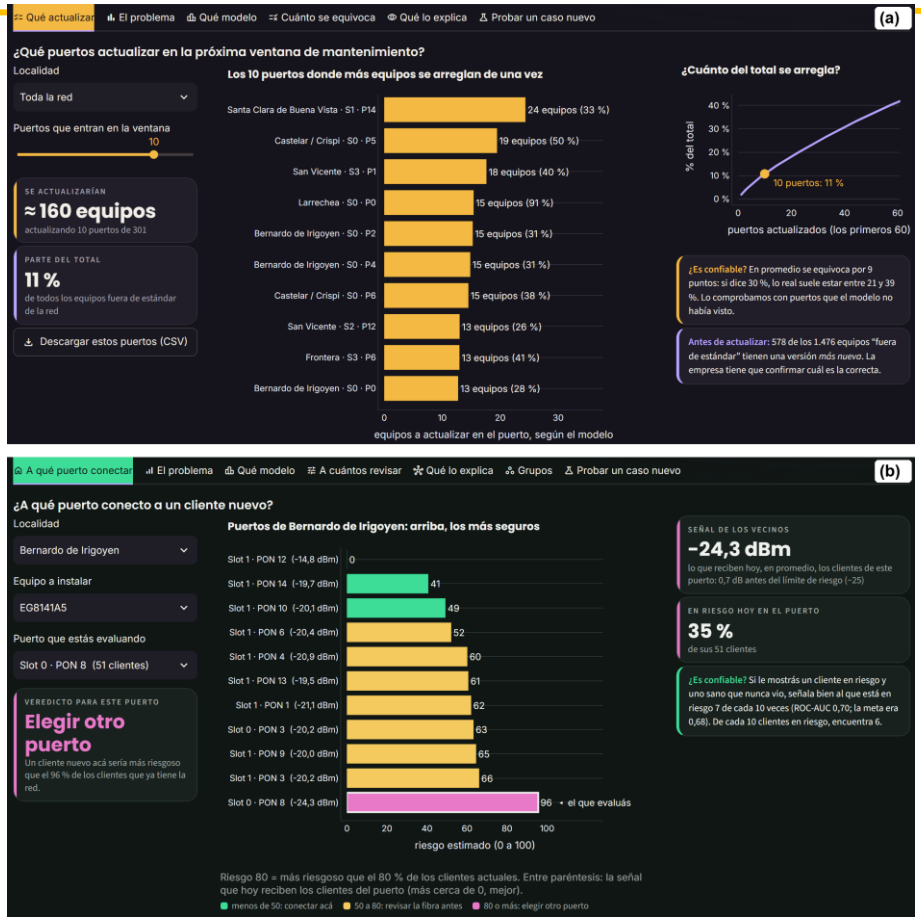


Fig. 9: Pantallas de decisión de la aplicación. (a) TP2: qué puertos actualizar en la próxima ventana de mantenimiento. (b) TP3: a qué puerto conectar a un cliente nuevo.

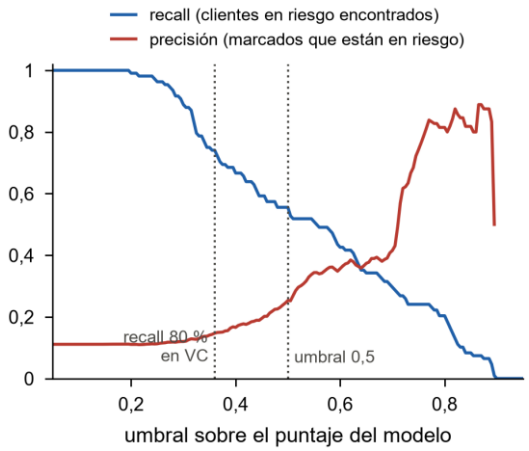


Fig. 8: Precisión y recall *de la regresión logística en prueba según el umbral (TP3). Las líneas punteadas marcan el umbral por defecto y el que da un* recall *del 80 % en validación cruzada.*

La búsqueda de hiperparámetros (Tabla 2) mejoró sobre todo a los modelos flexibles: *Gradient Boosting* redujo su RMSE de validación un 11,1 % en el TP2 y *Naive Bayes* sumó 0,16 puntos de PR-AUC en el TP3, casi siempre eligiendo más regularización (árboles menos profundos, hojas más grandes, más vecinos). La regresión logística (0,001 puntos) y *Ridge* casi no cambiaron: están limitados por la información disponible, no por sus hiperparámetros.

En la aplicación, cada modelo se traduce en una pantalla de decisión (Fig. 9), que se describe en la sección 3.5.

3.4 Agrupamiento

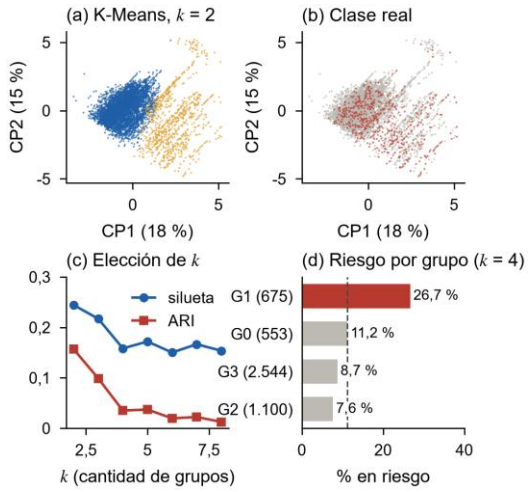


Fig. 10: Agrupamiento con K-Means *(TP3). (a) Grupos con* k *= 2 y (b) clase real (rojo: en riesgo) sobre las dos primeras componentes principales. (c) Silueta y ARI según* k*. (d) Porcentaje en riesgo por grupo con* k *= 4; la línea marca la tasa de la red.*

K-Means encontró una estructura débil (Fig. 10): la

mejor silueta fue 0,24, con $k = 2$, y el acuerdo con la clase real es bajo (ARI 0,16). Aun así, los grupos tienen sentido operativo. Con $k = 2$, un grupo de 853 clientes, con fibras más largas (mediana de 8,0 km contra 1,4 km) y vecinos con peor señal (−23,2 dBm contra −20,2 dBm), concentra el 26,3 % de clientes en riesgo, contra 8,1 % del resto. Con $k = 4$ aparece un grupo de 675 clientes con 26,7 % en riesgo, contra 7,6 a 11,2 % en los demás: son los tendidos largos, sobre todo de Bernardo de Irigoyen.

3.5 Aplicación desplegada

La Fig. 9 muestra las pantallas que responden las dos decisiones. En el apartado del TP2 (Fig. 9a), el usuario elige la localidad y cuántos puertos entran en la ventana de mantenimiento, y la aplicación ordena los puertos por la cantidad de equipos que se actualizarían según el modelo: con diez puertos se actualizarían unos 160 equipos, el 11 % de los 1.476 fuera de estándar, y la lista se puede descargar. En el apartado del TP3 (Fig. 9b), el técnico elige la localidad y el equipo a instalar, y la aplicación ordena los puertos según el riesgo que tendría un cliente nuevo, con un veredicto en palabras: «conectar acá», «revisar la fibra antes» o «elegir otro puerto». Los modelos serializados se cargan al iniciar y responden en el momento.

4. Discusión

4.1 Interpretación y comparación de los modelos

Los dos problemas supervisados llegan a una conclusión parecida por caminos distintos: en ambos, los modelos simples y regularizados igualan o superan a los ensambles, y la información que importa está en el puerto, no en el equipo individual. En el TP2, el *firmware* fuera de estándar se explica por la mezcla de modelos y la localidad, que reflejan decisiones de compra y de gestión; en el TP3, la señal se explica por la fibra compartida. Con 301 puertos y una foto estática, los ensambles tienen más capacidad que datos y terminan memorizando, como anticipa el compromiso entre sesgo y varianza [8]. Las referencias triviales fueron decisivas: el *Dummy* evitó celebrar una accuracy de 0,89 que no significa nada, y la regla de los vecinos mostró que la regresión logística, aunque cumple las metas, no agrega información sobre una regla que un técnico puede aplicar con una planilla. Es coherente con las revisiones [4], [5]: la calidad de un enlace depende sobre todo de mediciones del propio enlace.

La comparación entre ambos problemas también es metodológica. En la regresión, el R^2 ajustado mostró que sumar predictores con pocos puertos empeora el modelo aunque el R^2 suba; en la clasificación, el PR-AUC y la matriz de confusión mostraron lo que la accuracy escondía. En los dos casos, la forma de validar evitó estimaciones optimistas: con una partición aleatoria por cliente en el TP3, la señal de cada cliente de prueba habría entrado en el entrenamiento a través de la variable de sus vecinos [14].

4.2 Decisiones metodológicas y limitaciones

La etapa más desafiante no fue entrenar los modelos sino formular preguntas que los datos pudieran responder honestamente. El inventario es una foto estática y no permite predecir fallas: el primer planteo del TP3, predecir caídas con el listado de la OLT, no le ganó al azar y se reemplazó por la pregunta de la señal en riesgo. También costó detectar y cerrar las fugas: los atributos de *hardware* que dependen del modelo en el TP1, la versión de *firmware* en el TP2 y la potencia del propio equipo en el TP3. Si reiniciáramos el proyecto, pediríamos desde el principio lecturas periódicas de la OLT y el historial de reclamos, que permitirían un problema temporal real; validaríamos con la empresa la definición de *firmware* estándar, porque «fuera de estándar» no equivale a «desactualizado» (578 de los 1.476 equipos fuera de estándar, casi todos EG8143A5, tienen una generación de *firmware* mayor que la moda), y usaríamos la validación agrupada por puerto desde el TP1.

¿Es útil la aplicación para un técnico de la empresa? Para ordenar prioridades, sí: la lista de puertos a actualizar concentra el trabajo donde hay más equipos fuera de estándar, y el orden de puertos para un alta nueva evita conectar clientes donde los vecinos ya están al límite. Para decidir caso por caso, no: con una precisión de 0,25, tres de cada cuatro clientes marcados no están en riesgo, y el TP2 se equivoca en promedio en 8,8 puntos. Las limitaciones prácticas son tres. Primero, los datos de entrada: el TP3 necesita la señal actual de los vecinos, pero la aplicación usa la foto del 29/10/2026; se mitiga conectándola a lecturas periódicas de la OLT. Segundo, el alcance: ambos modelos aprendieron de una sola red y de un solo momento, y no deberían usarse en otra sin reentrenar; la aplicación ya avisa cuando un caso queda fuera del rango conocido. Tercero, la confiabilidad: el resultado se muestra con su margen de error y un veredicto en tres niveles, no como una certeza, y la decisión final es de un técnico.

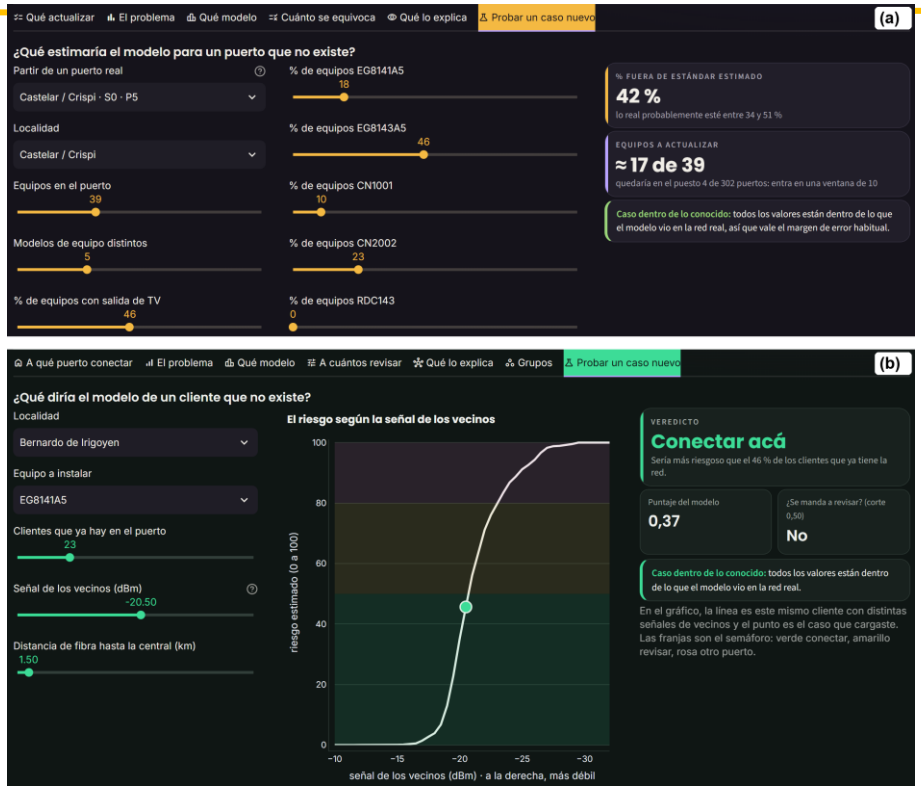


Fig. 11: Pestañas «Probar un caso nuevo», con validación de entradas. (a) TP2: estimación para un puerto definido por el usuario. (b) TP3: riesgo de un cliente nuevo según la señal de sus vecinos.

La validación de entradas es la principal defensa ante esos límites. Las pestañas «Probar un caso nuevo» (Fig. 11) permiten explorar casos que no existen, pero si un valor o una combinación queda fuera de lo que vio el modelo (por ejemplo, modelos que suman más de 100 % o una señal que ningún puerto tiene), la aplicación lo avisa en lugar de devolver un número sin sentido.

4.3 Hacia un entorno profesional

Llevar la solución a producción exigiría pasos que hoy no están [15]: un proceso automático que lea la OLT y el inventario, valide los datos y recalcule las variables; monitoreo de la deriva de las entradas y del desempeño, comparando las predicciones con las mediciones que llegan después de cada alta [20]; reentrenamiento periódico con control de versiones de datos y modelos; pruebas automáticas que, como ya hace el *pipeline*, bloqueen un modelo que no reproduce sus métricas, y una aplicación con autenticación alojada en la infraestructura de la empresa, porque los datos de la red son sensibles aunque se hayan quitado los números de serie y las IP. La escala no es un problema: la red tiene unos pocos miles de equipos y cada predicción toma milisegundos.

Los riesgos éticos son más sutiles. El riesgo de señal se concentra en los tendidos largos de algunas localidades; si la herramienta se usara para rechazar altas o para postergar inversiones en esas zonas, reforzaría una desigualdad de servicio entre clientes urbanos y rurales. Los puertos con pocos equipos y las localidades pequeñas tienen estimaciones menos confiables, y un error ahí afecta a clientes concretos. Antes de un despliegue a gran escala convendría documentar el modelo con una ficha que declare su alcance y sus límites [21], revisar el desempeño por localidad y usar la herramienta para priorizar mejoras de la fibra, no para excluir clientes.

5. Conclusiones

Este trabajo recorrió el ciclo completo de un proyecto de aprendizaje automático sobre el inventario de una red GPON real. El análisis exploratorio mostró que el límite del servicio está en los equipos de los clientes (97,2 % con Wi-Fi 4) y que el *firmware* fuera de estándar (28,9 %) se concentra en algunas localidades. Para estimarlo por puerto, *Ridge* alcanzó un R^2 de 0,59 y un error medio de 8,8 puntos en prueba, sin sobreajuste; para anticipar clientes con la señal en riesgo, la regresión logística logró un ROC-AUC de 0,70 y un PR-AUC de 0,39. Las metas se cumplieron, pero el hallazgo más útil es otro: una regla basada en la señal de los vecinos del puerto rinde lo mismo, lo que indica que el problema está en la fibra compartida y no en el módem. Además, el proyecto es reproducible de punta a punta: un solo comando regenera datos, modelos, aplicación y artículo, y cada cifra proviene del mismo archivo de métricas.

En lo práctico, la aplicación desplegada convierte esos modelos en dos decisiones concretas, qué

puertos actualizar primero y a qué puerto conectar a un cliente nuevo, con lenguaje simple, márgenes de error y avisos ante casos fuera de lo conocido. Sirve para ordenar prioridades, no para reemplazar la medición. Como trabajo futuro proponemos incorporar lecturas periódicas de la OLT y el historial de reclamos para estudiar la evolución de la señal y anticipar fallas, validar con la empresa la definición de *firmware* estándar, medir el desempeño de la herramienta en las altas reales y evaluar su equidad entre localidades.

- **Referencias**

[1] ITU-T, *Gigabit-capable passive optical networks (GPON): Physical media dependent (PMD) layer specification*, Rec. G.984.2, Amd. 1. Ginebra, Suiza: Unión Internacional de Telecomunicaciones, 2006.

[2] D. Zibar, M. Piels, R. Jones y C. G. Schäffer, "Machine learning techniques in optical communication", *J. Lightw. Technol.*, vol. 34, no. 6, pp. 1442–1452, mar. 2016.

[3] F. N. Khan, Q. Fan, C. Lu y A. P. T. Lau, "An optical communication's perspective on machine learning and its applications", *J. Lightw. Technol.*, vol. 37, no. 2, pp. 493–516, ene. 2019.

[4] F. Musumeci *et al.*, "An overview on application of machine learning techniques in optical networks", *IEEE Commun. Surveys Tuts.*, vol. 21, no. 2, pp. 1383–1408, 2019.

[5] J. Mata *et al.*, "Artificial intelligence (AI) methods in optical networks: A comprehensive survey", *Opt. Switch. Netw.*, vol. 28, pp. 43–57, abr. 2018.

[6] D. Rafique y L. Velasco, "Machine learning for network automation: Overview, architecture, and applications", *J. Opt. Commun. Netw.*, vol. 10, no. 10, pp. D126–D143, oct. 2018.

[7] A. E. Hoerl y R. W. Kennard, "Ridge regression: Biased estimation for nonorthogonal problems", *Technometrics*, vol. 12, no. 1, pp. 55–67, feb. 1970.

[8] T. Hastie, R. Tibshirani y J. Friedman, *The Elements of Statistical Learning*, 2.ª ed. Nueva York, EE. UU.: Springer, 2009.

[9] L. Breiman, "Random forests", *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, oct. 2001.

[10] J. H. Friedman, "Greedy function approximation: A gradient boosting machine", *Ann. Statist.*, vol. 29, no. 5, pp. 1189–1232, oct. 2001.

[11] H. He y E. A. Garcia, "Learning from imbalanced data", *IEEE Trans. Knowl. Data Eng.*, vol. 21, no. 9, pp. 1263–1284, sep. 2009.

[12] T. Saito y M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets", *PLoS ONE*, vol. 10, no. 3, art. e0118432, mar. 2015.

[13] S. Kaufman, S. Rosset, C. Perlich y O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance", *ACM Trans. Knowl. Discov. Data*, vol. 6, no. 4, art. 15, dic. 2012.

[14] D. R. Roberts *et al.*, "Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure", *Ecography*, vol. 40, no. 8, pp. 913–929, ago. 2017.

[15] D. Sculley *et al.*, "Hidden technical debt in machine learning systems", en *Proc. Adv. Neural Inf. Process. Syst.*, vol. 28, 2015, pp. 2503–2511.

[16] E. Breck, S. Cai, E. Nielsen, M. Salib y D. Sculley, "The ML test score: A rubric for ML production readiness and technical debt reduction", en *Proc. IEEE Int. Conf. Big Data*, 2017, pp. 1123–1132.

[17] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python", *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[18] P. J. Rousseeuw, "Silhouettes: A graphical aid to the interpretation and validation of cluster analysis", *J. Comput. Appl. Math.*, vol. 20, pp. 53–65, nov. 1987.

[19] Streamlit Inc. (2026). *Streamlit documentation* [En línea]. Disponible: https://docs.streamlit.io

[20] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy y A. Bouchachia, "A survey on concept drift adaptation", *ACM Comput. Surv.*, vol. 46, no. 4, art. 44, abr. 2014.

[21] M. Mitchell *et al.*, "Model cards for model reporting", en *Proc. Conf. Fairness, Accountability, and Transparency (FAT*)*, 2019, pp. 220–229.